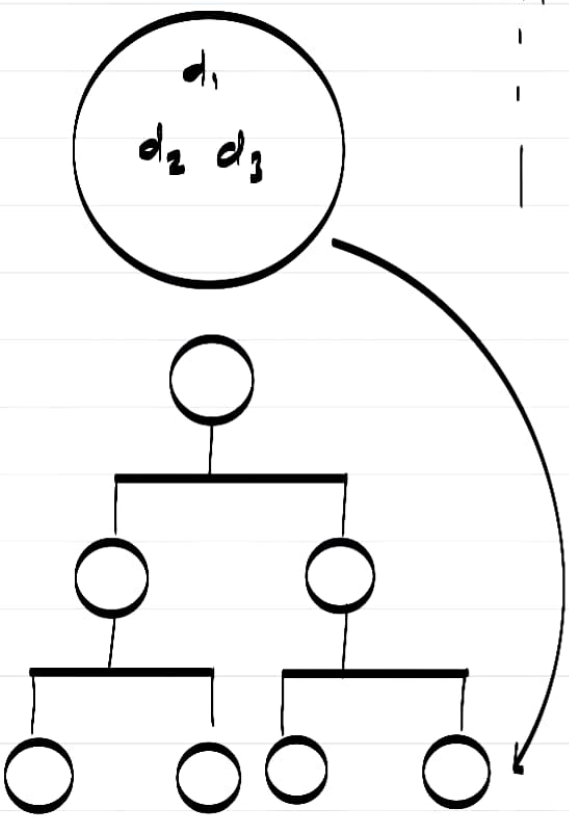
Machine Learning

27-04-26

Decision Tree Regression

- In DT regr. best decision rule for split is found to minimize the MSE - mean squared error



x_1	x_2	x_3	y_{act}	y_{pred}
1	1	1	$2 - y_1$	$3 - \hat{y}_1$
1	1	1	$4 - y_2$	$3 - \hat{y}_2$
1	1	1	$3 - y_3$	$3 - \hat{y}_3$

$$\hat{y} = \bar{y} = avg(y_1\ y_2\ y_3)$$

$$= \frac{2+4+3}{3} = 3$$

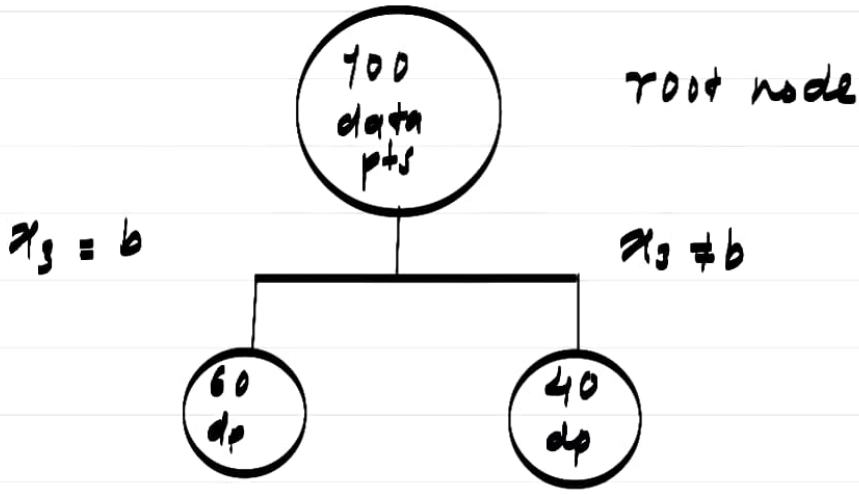
$$mse = \frac{1}{3} \sum_{i=1}^{3} (y_i - \hat{y}_i)^2$$

$$= \frac{1}{3}\left[(2-3)^2 + (4-3)^2 + (3-3)^2\right]$$

$$= 2/3$$

At any given level, decision rule giving maximum MSE reduction is the best decision rule.

reduction in MSE = mse of parent node − weighted sum of child nodes



for parent node:

$$\bar{y} = \frac{1}{100}\sum_{i=1}^{100} y_i = \hat{y}_{parent} = \hat{y}_p$$ → y_{pred} for all 100 datapts

$$\text{mse of parent} = \frac{1}{100}\sum_{i=1}^{100}(y_i - \hat{y}_p)^2 = mse_{parent}$$

for left child

$$\bar{y} = \frac{1}{60}\sum_{i=1}^{60} y_i = \hat{y}_{child1} = \hat{y}_{c_1}$$ — y_{pred} for all 60 dtps

$$\text{mse of child1} = \frac{1}{60}\sum_{i=1}^{60}(y_i - \hat{y}_{c_1})^2 = mse_{child1}$$

for right child

$$\bar{y} = \frac{1}{40}\sum_{i=1}^{100} y_i = \hat{y}_{child2} = \hat{y}_{c_2}$$

$$\text{mse of child2} = \frac{1}{40}\sum_{i=1}^{40}(y_i - \hat{y}_{c_2})^2 = mse_{child2}$$

reduction in mse : $mse_{parent} - \left[\frac{60}{100} \times mse_{child1} + \frac{40}{100} \times mse_{child2}\right]$

	x_1	x_2	Y
d_1	10	4	1
d_2	12	10	2
d_3	21	5	5
d_4	25	12	8

root node

DT regr. has same hyperparameters as that of DT clf.

(except criterion)

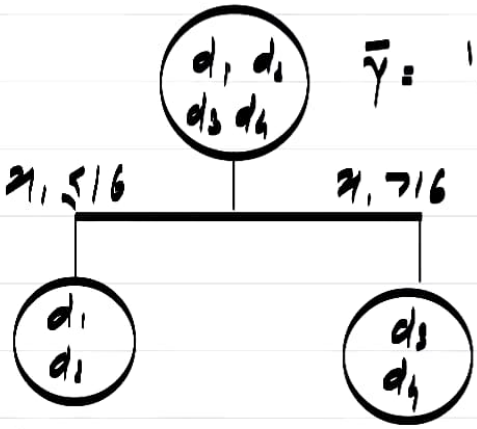
lets say

max features: 2

x_1 and x_2 are picked - both are continuous

$\downarrow$ 100 rules $\quad$ $\downarrow$ 100 rules $\longrightarrow$ 200 rules to test

dec rule 1



$\bar{y} = \frac{1+2+5+8}{4} = 4$

$\bar{y} = \frac{1+2}{2} = 1.5$ $\qquad$ $\bar{y} = \frac{5+8}{2} = 6.5$

$\hat{y}_p = 4$

$\hat{y}_{c1} = 1.5$

$\hat{y}_{c2} = 6.5$

$$\text{mse of parent node} = \frac{1}{4}\sum_{i=1}^{4}(y_i - \hat{y}_p)^2$$

$$= \frac{1}{4}\left[(1-4)^2 + (2-4)^2 + (5-4)^2 + (8-4)^2\right]$$

$$= \frac{30}{4} = 7.5$$

mse for child 1 = $\frac{1}{2} \sum_{i=1}^{2} (y_i - \hat{y}_{c1})^2$

$= \frac{1}{2} \left[(1-1.5)^2 + (2-1.5)^2 \right]$

$= 0.25$

mse for child 2 : $\frac{1}{2} \sum_{i=1}^{2} (y_i - \hat{y}_{c2})^2$

$= \frac{1}{2} \left[(5-6.5)^2 + (8-6.5)^2 \right]$

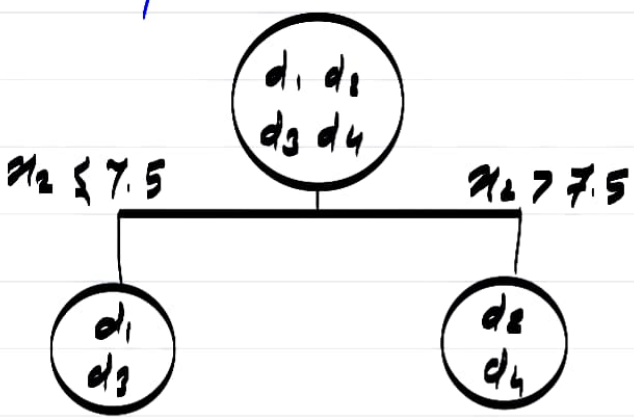
$= 2.25$

red^n in mse = mse parent $- \left[\frac{2}{4} \text{ mse child1} + \frac{2}{4} \times \text{mse child2} \right]$

$= 7.5 - \left[\frac{2}{4} \times 0.25 + \frac{2}{4} \times 2.25 \right]$

$= 7.5 - 1.25 = 6.25$

Similarly for dec rule 2



red^n in mse (for dec rule 2) = 1.2

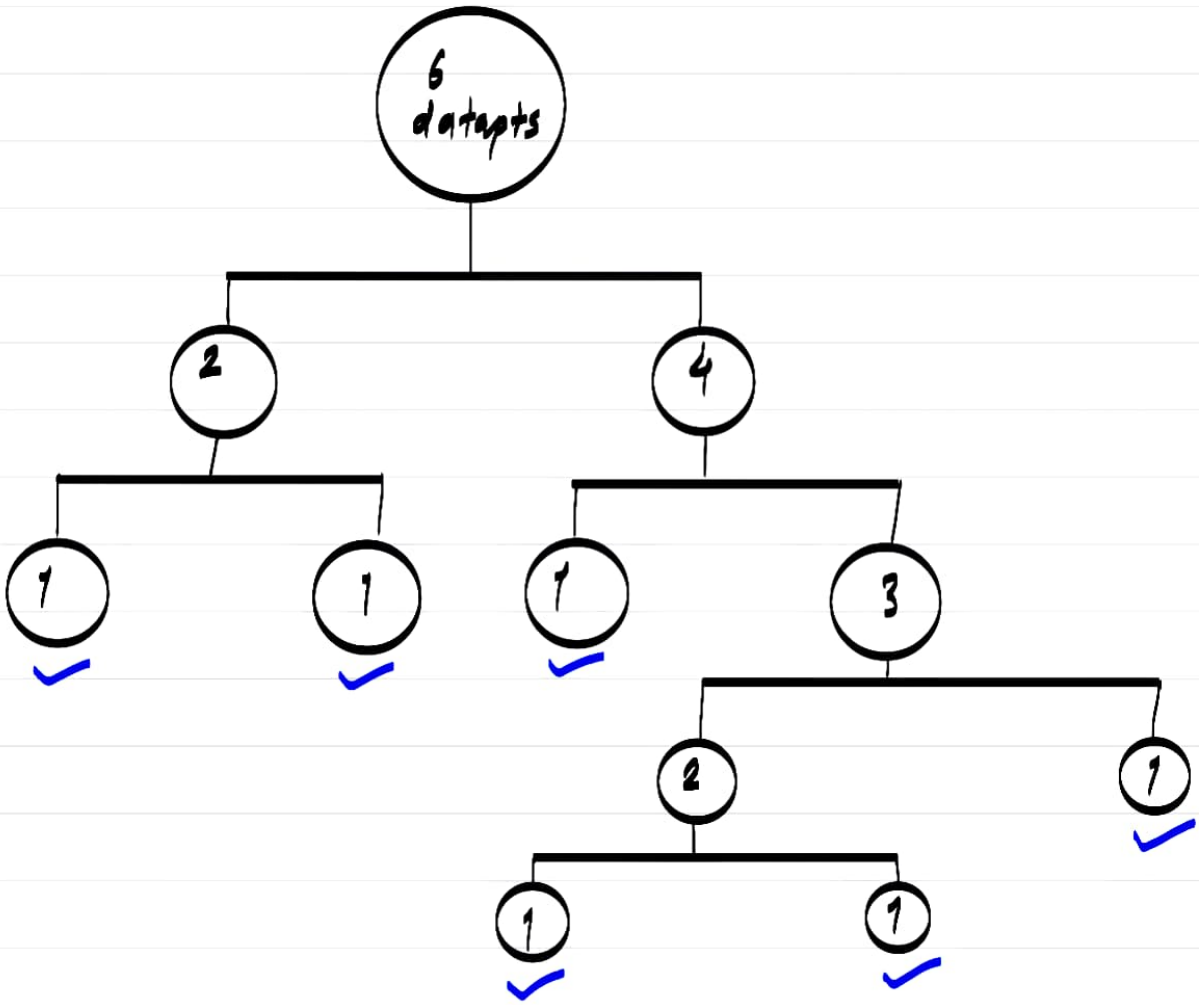
dec rule 1 is better than dec rule 2 as
red^n in mse - dec rule 1 > red^n in mse - dec rule 2

max depth = None

min samples split = 2

min samples leaf = 1

- when this is set, tree can grow upto <u>any depth</u>



leaf node: nodes having no child

here, every leaf node has a single datapoint making mse of every leaf node = 0

5 $\bar{y} = 5$ $mse = \frac{1}{7}(5-5)^2 = 0$

total mse of this DT will become <u>zero</u>

Due to above hyperparameter settings, DTree will grow deep and eventually makes train mse = 0

- train mse = 0 means the model is 100% accurate on train data

- when mse = 0 then this model will give very bad performance on test data.

This issue is known as overfitting

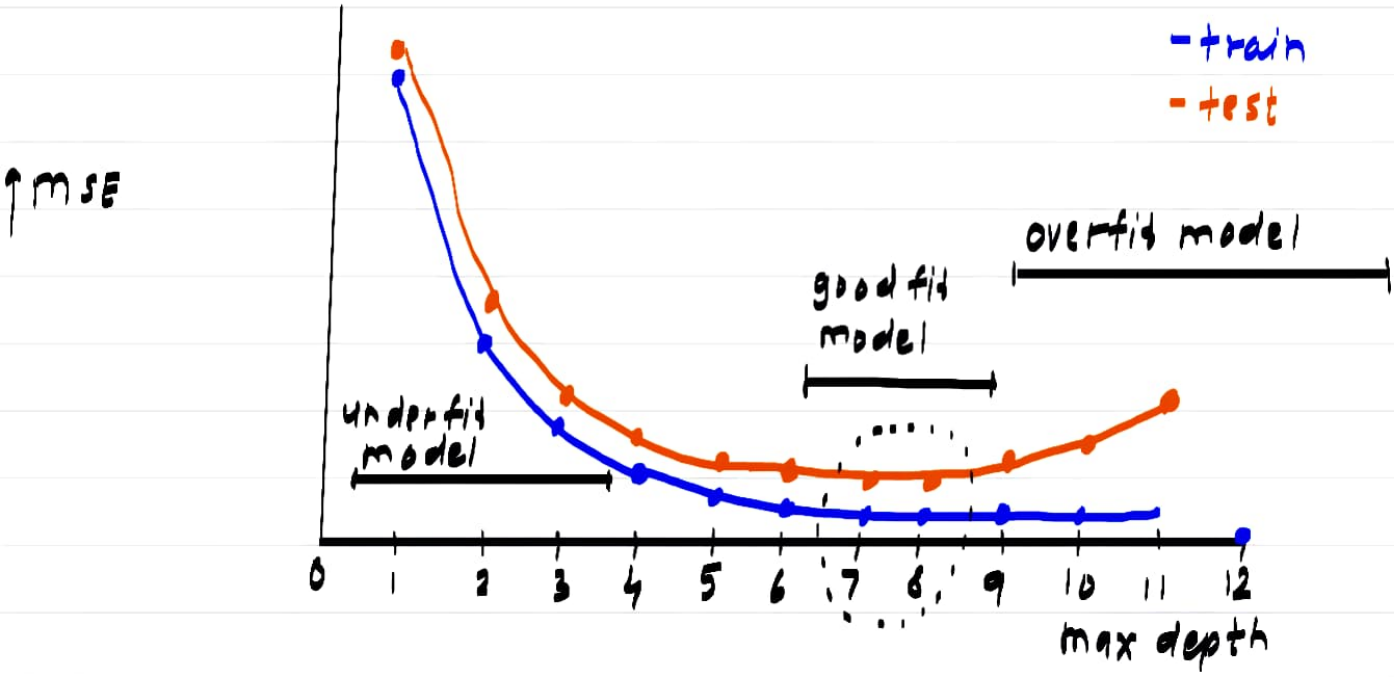
overfitting: the train mse of model is too low that it gives very poor performance on test data.

learning to ride bike ⟶ trained on colony roads

hyperparameters:
duration : unlimited
roads : only colony roads

rider will become 100% accurate on colony roads but
will eventually fail badly on any other road

- this is overfitting in real life.



underfit : bad performance on train and test data

good / best fit: good performance on train and test data

overfit : good performance on train data and bad performance on test data

- Always avoid overfitting and underfitting in every ML model training

* How do we avoid overfitting / underfitting?

- in above case, overfiting is occuring due to improper values of hyperparameters, hence hyperparameter tuning will solve overfiting

What are causes of overfiting?

1) Improper values of hyperparameters
2) Too many features in training

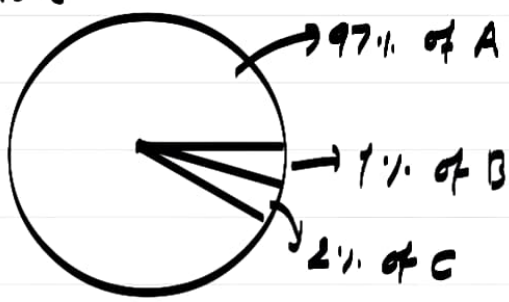
   model 1 → 5 features

   model 2 → 20 features

   there are high chances of overfiting in model 2

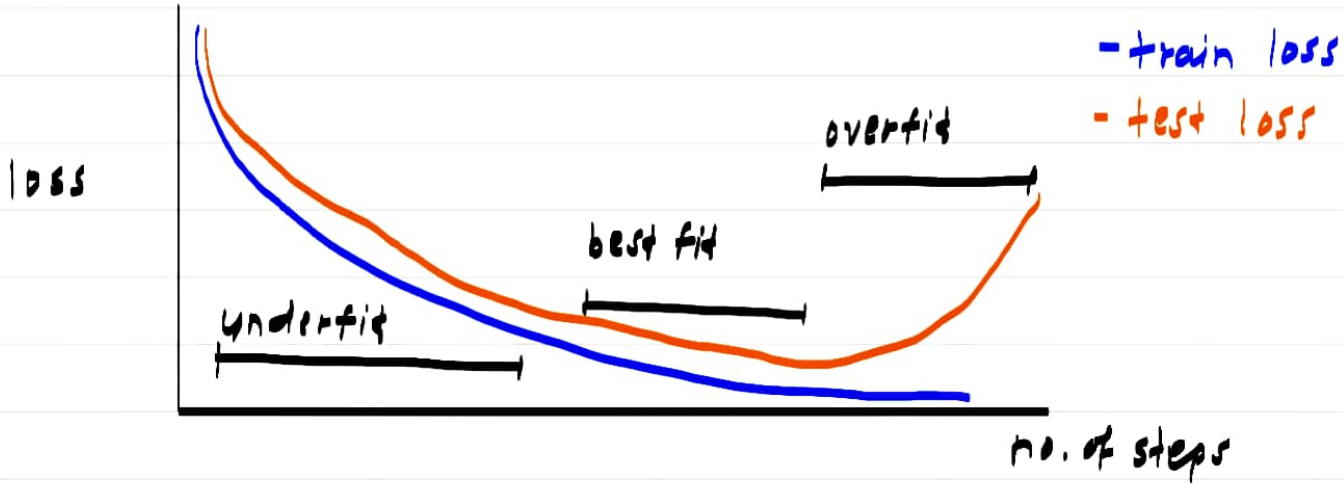
3) class imbalance issue

   eg. targed variable



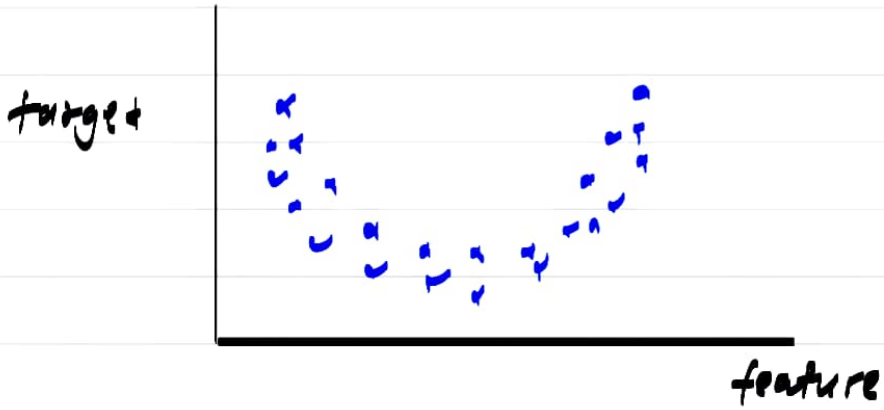
   model gets overfitted on class A and gives bad performance of classes B and C

4) Too many model parameter updation steps during gradient descent



what causes under fiting?

1) Not choosing correct model / algorithm for given problem statement



choosing linear regr. here will cause under fiting

2) Improper hyperparameter values

3) Data quality issue - bad data, missing / wrong values

4) Not enough steps in gradient descent